

DECISION-MAKING ANALYSIS AND PROOF OF CONCEPT

MastitisGuard AI

Team Name: H14_SPIRIT

Team ID: 146962

Problem Statement Number: 26109

Journey stage: Built solution

1. Introduction

Building MastitisGuard AI required choosing between real alternatives at almost every layer of the system — which algorithm to train, how to structure the hardware, how to handle sensors that were not yet available, and how the model should improve after deployment. This document records those decisions explicitly, along with what has actually been built and verified versus what remains designed but not yet implemented. The build-status table in Section 4 uses the same terms — built, bench-tested, budgeted, not yet built — as the Bill of Materials (Section 6) and the ML Model Documentation (Section 5), so a reviewer sees one consistent status for every component across all three documents.

2. Purpose of This Document

To give a single, honest reference for two related questions a reviewer will naturally ask: "why did you choose this approach over the alternatives," and "how much of what you're describing actually exists right now." Section 3 answers the first; Section 4 answers the second.

3. Key Decisions and Alternatives Considered

Decision	Alternatives Considered	Why This Option Was Chosen
Predictive engine	Deterministic baseline only vs. supervised ML only vs. both, layered	Both, layered — the rolling-baseline deviation layer works from day one with zero training data; a CatBoost classifier trained on top of it (Validation document, Section 3) sharpens the signal into a calibrated risk tier once trained. Neither layer alone was judged sufficient: baseline-only gives no risk tiering, ML-only would have no signal until a model exists.
Model algorithm	CatBoost vs. XGBoost / Random Forest / Neural Network	CatBoost — native categorical handling + built-in C++ exporter enables offline ESP32 deployment without a separate conversion pipeline.
Baseline strategy	Per-cow rolling baseline vs. herd-wide static threshold	Per-cow — accounts for natural individual variation in temperament and physiology; avoids both false alarms and missed detections (Validation document, Section 3).
Training inputs	Include CMT/vision as a feature vs. exclude it	Exclude — CMT only turns positive once mastitis is already clinical; including it would let the model trivially echo a confirmation test instead of learning genuine early warning.

Decision	Alternatives Considered	Why This Option Was Chosen
Inter-node communication	ESP-NOW vs. Wi-Fi/cloud vs. LoRaWAN/Bluetooth alone	ESP-NOW — peer-to-peer, no router or internet required, low power draw; directly satisfies the rural-connectivity constraint. LoRaWAN/BLE remain optional add-ons for longer-range or app-facing telemetry (Bill of Materials, Section 2).
Physical architecture	Three separate units vs. one combined device	Three units — matches where each measurement physically happens (on the animal, at the milking point, and centrally); see Validation document, Section 3, final row.
Handling missing sensors during prototyping	Delay the build until every sensor is sourced vs. build now with clearly flagged placeholders	Build now with placeholders — DHT11 stands in for skin-contact temperature during bench testing (MAX30205 is the specified pilot component), and missing pH readings fall back to the cow's own rolling baseline rather than a fabricated number. This allowed the full pipeline to be demonstrated end-to-end without blocking on procurement.
Class imbalance handling	Class weighting vs. synthetic oversampling (e.g. SMOTE)	Class weighting — simpler, and avoids compounding synthetic noise on top of an already-synthetic dataset; oversampling becomes more appropriate once real data is available.
Fusion weight between sensors and vision/CMT	Fixed 70/30 split (original problem brief) vs. data-tuned weight	Data-tuned — a validation sweep finds the AUC-optimal blend rather than assuming one, which is what surfaced the fusion-weight finding described in the ML Model Documentation.
Model improvement over time	On-device online learning vs. periodic vet-confirmed retraining	Periodic retraining — an ESP32 cannot practically retrain a gradient-boosted model on-device, and on-device learning cannot incorporate the vet's confirmed diagnosis, which is the actual ground truth needed to improve the model responsibly.

4. Proof-of-Concept Status

A candid inventory of what is built and verified versus what is designed but not yet implemented. Status labels match the Bill of Materials (Section 6) exactly.

Component	Status	Notes
Unique Cow ID / barcode identification	Designed and specified	Architecture defined; ear-tag scanning via ESP32-CAM tested for ID scan, not yet field-trialled
Wearable sensing — activity, rumination proxy, mobility (GPS), temperature	Built and bench-tested	MPU6050, NEO-6M GPS wired and verified on ESP32 hardware; temperature currently bench-tested via a DHT11 stand-in, with MAX30205 specified and costed for the pilot build (Bill of Materials, Section 2)
Milk unit sensing — temperature, yield	Built and bench-tested	DS18B20 and HX711 + load cell wired and verified; calibration procedure documented
Milk unit — pH sensing	Budgeted, not yet physically integrated	Placeholder logic implemented in firmware; physical probe budgeted (Bill of Materials, Section 3) and scheduled for Phase 1 pilot integration

Component	Status	Notes
Milk unit — automated CMT vision	Not yet built	ESP32-CAM module specified in architecture and used today for ID scan; CMT vision model not yet developed
Shed ESP central inference — baseline layer	Built and verified	Rolling 7–14 day median and deviation-residual computation runs fully offline on the Shed ESP
Shed ESP central inference — CatBoost layer	Built and verified, on synthetic data	CatBoost model trained and evaluated on synthetic data (ROC-AUC 0.964; ML Model Documentation, Section 5), exported to C++, and confirmed to run offline on the ESP32; field accuracy on real cows is the Phase 1 milestone, not yet demonstrated
Mobile App & Edge Client	Built, bundled as Android APK, verified	Developed in TypeScript and React Native, bundled as an Android APK using the Expo toolchain; verified against the Shed ESP's local telemetry
Self-learning feedback loop	Designed, not yet built	Requires a live farm deployment generating vet-confirmed outcomes to begin operating (ML Model Documentation, Section 11)

5. Feasibility and Viability

5.1 Why the Idea Is Technically Feasible

- ESP32 and the available sensors can collect cow health and milk data, while ESP-NOW enables local data transfer without dependence on internet connectivity.
- The Shed ESP can combine these parameters with each cow's historical baseline to generate a mastitis-risk score locally, without a cloud round-trip.
- Each module — wearable, milk measurement, and Edge AI — can be developed and tested separately before integration, which is exactly how this prototype was built and verified (Section 4).

5.2 Potential Challenges, Risks, and Mitigation Strategies

Challenge	Risk	Mitigation Strategy
Sensor reliability	Cow movement, sensor placement, and environmental conditions may cause inaccurate or inconsistent readings.	Proper sensor placement, protective casing, and repeated measurements to filter out noisy or abnormal readings.
Model accuracy under real conditions	The CatBoost model's 0.964 AUC is a synthetic-data result; limited farm-specific training data may lead to false positives or false negatives on real cows.	Collect cow-specific real data through the Phase 1 pilot, retrain on vet-confirmed outcomes, and validate the risk model against real observations before any real-world accuracy claim is made — this is the Phase 1 self-learning loop described in the ML Model Documentation.
System integration	Maintaining reliable Cow ID tracking and wireless communication between the wearable, milk unit, and Shed ESP can be challenging in a real farm environment.	Use unique Cow IDs, data validation, and reliable ESP-NOW communication with retry mechanisms to prevent data mismatch or loss.

6. Conclusion

Every major decision in this project was made against at least one concrete alternative, for a reason that can be stated and defended rather than assumed. Equally, this document does not claim more than has actually been built: the sensing, communication, baseline computation, and offline CatBoost inference layers are implemented and bench-verified on synthetic data; the pH probe, automated CMT vision, and self-learning retraining loop are designed and specified, but their validation depends on the real-farm pilot that is the project's next concrete step — the same Phase 1 milestone described in the Market Research document, Section 4, and the ML Model Documentation, Section 11.

7. References

- Espressif Systems. ESP-NOW — ESP-IDF Programming Guide. docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html.
- Ranjan, R., Gupta, M.K., & Singh, K.K. (2011). Study of bovine mastitis in different climatic conditions in Jharkhand, India. *Veterinary World*, 4(5), 205–208.